

```

1  module PISO (
2      input wire SCLK,                // System clock
3      input wire CLR,                // Clear signal
4      input wire LOAD,                // Load 40-bit parallel word
5      input wire EN,                 // Enable
6
7      input wire [39:0] PDATAIN_L,   // 40-bit parallel input data - Left
8      input wire [39:0] PDATAIN_R,   // 40-bit parallel input data - Right
9
10     output reg DATAOUT_L,          // Serial output - Left (Single-bit output)
11     output reg DATAOUT_R          // Serial output - Right (Single-bit output)
12 );
13
14     reg [39:0] shift_reg_L;
15     reg [39:0] shift_reg_R;
16     reg [5:0] bit_cnt;
17
18     always @(posedge SCLK or posedge CLR) begin
19         if (CLR) begin
20             shift_reg_L <= 40'b0;
21             shift_reg_R <= 40'b0;
22             bit_cnt <= 6'b0;
23             DATAOUT_L <= 1'b0;
24             DATAOUT_R <= 1'b0;
25         end else if (LOAD) begin
26             shift_reg_L <= PDATAIN_L;
27             shift_reg_R <= PDATAIN_R;
28             bit_cnt <= 6'b101000; // 40 in decimal
29         end else if (EN && bit_cnt != 0) begin
30             DATAOUT_L <= shift_reg_L[39]; // Output MSB of the shift register
31             DATAOUT_R <= shift_reg_R[39]; // Output MSB of the shift register
32             shift_reg_L <= shift_reg_L << 1; // Shift left to push MSB out
33             shift_reg_R <= shift_reg_R << 1; // Shift left to push MSB out
34             bit_cnt <= bit_cnt - 1;
35         end
36     end
37
38 endmodule
39
40
41 //Testbench for PISO
42 `timescale 1ns / 1ps
43
44 module PISO_tb;
45
46     // Inputs
47     reg SCLK;
48     reg CLR;
49     reg LOAD;
50     reg EN;
51     reg [39:0] PDATAIN_L;
52     reg [39:0] PDATAIN_R;
53
54     // Outputs
55     wire DATAOUT_L;
56     wire DATAOUT_R;
57
58     // Internal capture registers (Corrected to 40 bits wide)
59     reg [39:0] captured_L;
60     reg [39:0] captured_R;
61
62     integer i;
63
64     // Instantiate the module under test
65     PISO uut (
66         .SCLK(SCLK),
67         .CLR(CLR),
68         .LOAD(LOAD),
69         .EN(EN),

```

```

70     .PDATAIN_L(PDATAIN_L),
71     .PDATAIN_R(PDATAIN_R),
72     .DATAOUT_L(DATAOUT_L),
73     .DATAOUT_R(DATAOUT_R)
74 );
75
76 // Clock generation
77 initial begin
78     SCLK = 0;
79     forever #5 SCLK = ~SCLK; // 10ns clock period
80 end
81
82 // Stimulus and capture logic
83 initial begin
84     // Init
85     CLR = 1;
86     LOAD = 0;
87     EN = 0;
88     PDATAIN_L = 40'hFFFFFFB4380;
89     PDATAIN_R = 40'h0002546D5C;
90
91     captured_L = 40'b0;
92     captured_R = 40'b0;
93
94     #12;
95     CLR = 0;
96
97     #10;
98     LOAD = 1;
99     #10;
100    LOAD = 0;
101
102    EN = 1;
103
104    // Capture 40 serial bits
105    for (i = 0; i < 40; i = i + 1) begin
106        @(posedge SCLK);
107        captured_L = {captured_L[38:0], DATAOUT_L}; // Shift right, capturing MSB
108        captured_R = {captured_R[38:0], DATAOUT_R}; // Shift right, capturing MSB
109    end
110
111    EN = 0;
112
113    // Display result
114    $display("\nGenerated Serial Output:");
115    $display("OutputL = %010h", captured_L);
116    $display("OutputR = %010h", captured_R);
117
118    // Expected values for comparison
119    $display("ExpectedL = %010h", 40'hFFFFFFB4380);
120    $display("ExpectedR = %010h", 40'h0002546D5C);
121
122    if (captured_L == 40'hFFFFFFB4380 && captured_R == 40'h0002546D5C)
123        $display("Compare Result: PASS");
124    else
125        $display("Compare Result: FAIL");
126
127    $stop;
128 end
129
130 endmodule
131

```